

THIAGARAJAR COLLEGE OF ENGINEERING

Madurai – 625 015

DEPARTMENT OF INFORMATION TECHNOLOGY

Bachelor of Technology – Information Technology

DWELLIO

Rental Management System

Modern PropTech Platform for Tenants, Property Owners, and Platform Administrators

Submitted by

S.No	Name of the Student	Register Number
1	Sujitaa A	[2303917720522160]
2	Aadhithya A	[2303917720521301]

Under the Guidance of

Ms.J.Jeyasoundari

Department of Information Technology

Academic Year 2026 – 2027

September 2026

ABSTRACT

The residential rental real estate industry suffers from operational friction: scattered listings without identity verification, slow manual applicant screening, offline check-based rent collection, untracked repair requests, and opaque communication between tenants and landlords.

Dwellio (developed under the Nestora platform specification) is an end-to-end, production-style Software-as-a-Service (SaaS) web application built on a decoupled MERN stack (React 18, Node.js/Express and MongoDB). It unites tenants, property owners and platform administrators in a single digital workspace. Key capabilities include a two-tier listing moderation pipeline, single-click lease binding, a simulated rent payment gateway with receipts, and a three-step maintenance ticketing timeline, all secured through JSON Web Token (JWT) authentication, bcrypt password hashing and role-based access control.

The system was verified through a client build, database seeding and endpoint tests; all eight verification modules passed.

Keywords: MERN Stack, PropTech, Rental Management, Role-Based Access Control, JWT Authentication, MongoDB

TABLE OF CONTENTS

1. Introduction.....	1
1.1 Overview	1
1.2 Stakeholders.....	1
1.3 Project Specification.....	1
2. Problem Statement & Comparison with Legacy Systems.....	2
2.1 Problem Statement.....	2
2.2 Comparison with Legacy Systems	2
3. System Architecture & Technology Stack.....	3
3.1 Presentation Tier (Frontend).....	3
3.2 Application Tier (Backend API).....	3
3.3 Persistence Tier (Database)	3
3.4 Security & Cryptography Tier.....	3
4. Role-Based Access Control (RBAC) Matrix.....	4
5. Database Schema & Data Models	5
5.1 User Schema	5
5.2 Property Schema.....	5
5.3 RentalRequest Schema	6
5.4 RentPayment Schema	6
5.5 MaintenanceRequest Schema	6
5.6 Review Schema	7
6. Key Operational Workflows	8
6.1 Listing Moderation Pipeline	8
6.2 Single-Click Lease Binding.....	8
6.3 Simulated Rent Payment Gateway	8
6.4 Three-Step Maintenance Timeline	8
7. Verification & Automated Test Results.....	9
8. Demo Evaluation Accounts	10
9. Conclusion	11

LIST OF TABLES

Table 1.1: Project Specification Summary	1
Table 2.1: Legacy Approach vs. Dwello Solution	2
Table 4.1: Role-Based Access Control Matrix	4
Table 5.1: User Schema	5
Table 5.2: Property Schema.....	5
Table 5.3: RentalRequest Schema	6
Table 5.4: RentPayment Schema.....	6
Table 5.5: MaintenanceRequest Schema	6
Table 5.6: Review Schema.....	7
Table 7.1: Verification and Automated Test Results.....	9
Table 8.1: Demo Evaluation Accounts	10

1. INTRODUCTION

1.1 Overview

Dwellio (developed under the Nestora platform specification) is an end-to-end, production-style Software-as-a-Service (SaaS) web application designed to bring the entire residential rental lifecycle – from listing and application to lease management, rent payment and maintenance – into one seamless digital workspace.

1.2 Stakeholders

The platform unites three essential stakeholder groups:

- **Tenants:** Search verified properties, apply online, track application statuses, manage active leases, execute simulated rent payments with receipts, and log photographic maintenance requests with real-time status timelines.
- **Property Owners (Landlords):** Publish residential properties, manage media uploads, review and approve applicant inquiries, toggle availability, track rent ledgers, and dispatch contractors for maintenance.
- **Platform Administrators:** Moderate submitted listings via an approval queue, administer user accounts (enable/disable), oversee network repair tickets, and analyse KPIs using interactive Recharts data visualisations.

1.3 Project Specification

Table 1.1 summarises the key specification details of the system.

Table 1.1: *Project Specification Summary*

Attribute	Details
Architecture	Decoupled MERN Stack (React 18 + Node/Express + MongoDB)
Alternative Identifier	Nestora Rental Management Platform
Date of Release	September 2026
Version Status	Version 1.0.0 (Production Verified)

2. PROBLEM STATEMENT & COMPARISON WITH LEGACY SYSTEMS

2.1 Problem Statement

The residential rental real estate industry has historically suffered from acute operational friction: scattered listings without identity verification, slow manual applicant screening, offline check-based rent collection, untracked repair requests, and opaque communication between tenants and landlords. Dwellio is engineered to remove these pain points through a structured, role-based digital platform.

2.2 Comparison with Legacy Systems

Table 2.1 contrasts the legacy or manual approach to each market challenge with the corresponding architectural solution implemented in Dwellio.

Table 2.1: *Legacy Approach vs. Dwellio Solution*

Market Challenge	Legacy / Manual Approach	Dwellio Architectural Solution
Listing Quality Control	Unverified listings, outdated prices, ghost properties	Mandatory two-tier moderation: properties stay 'pending' until Admin review.
Lease Binding	Paper contracts, manual deposit handoffs, delayed notices	Single-click approval: automatically rents property, binds tenant, rejects rivals.
Rent Collection	Paper checks, disparate bank transfers, lost receipts	Simulated digital checkout generating unique mock transaction IDs & receipts.
Maintenance Accountability	Lost SMS / phone calls, lack of photos, no timeline	Structured ticketing with category, priority, photos, and 3-step visual status timeline.

3. SYSTEM ARCHITECTURE & TECHNOLOGY STACK

Dwellio is engineered following a decoupled, four-tier web architecture that ensures modularity, security and scalability. The responsibilities of each tier are described below.

3.1 Presentation Tier (Frontend)

A React 18 Single Page Application bundled with Vite. It utilises React Router v6 for client-side routing, Lucide React for consistent iconography, Recharts for data analytics, and a custom CSS design system.

3.2 Application Tier (Backend API)

A Node.js and Express.js RESTful API engine providing centralised route controllers, error handling, CORS headers, and Multer-based multipart/form-data image handling.

3.3 Persistence Tier (Database)

A MongoDB document database operated through the Mongoose ODM, utilising declarative schema validations, compound unique indexes, and pre-save hooks.

3.4 Security & Cryptography Tier

Stateless 30-day JSON Web Tokens (JWT) signed with HMAC-SHA256 handle authentication, alongside bcrypt password hashing with 10 salt rounds.

4. ROLE-BASED ACCESS CONTROL (RBAC) MATRIX

Access to platform features is governed by the role assigned to each account (tenant, owner or admin). Table 4.1 summarises the permissions granted to each role, along with public (unauthenticated) access.

Table 4.1: *Role-Based Access Control Matrix*

Platform Feature / Route	Public	Tenant	Owner	Admin
Browse Catalog & Details (/properties)	Yes	Yes	Yes	Yes
Submit Rental Inquiry (/api/rentals)	No	Yes	No	No
Tenant Portal (/tenant/*)	No	Yes	No	No
Owner Dashboard & Listings (/owner/*)	No	No	Yes	Yes
Approve / Reject Lease Requests	No	No	Yes	Yes
Admin Moderation Queue (/admin/*)	No	No	No	Yes

5. DATABASE SCHEMA & DATA MODELS

The persistence layer is implemented with MongoDB and Mongoose. The data model comprises six collections – User, Property, RentalRequest, RentPayment, MaintenanceRequest and Review – described in the following sections.

5.1 User Schema

Stores platform accounts, credentials, and roles.

Table 5.1: *User Schema*

Field Name	Data Type	Constraints & Description
name	String	Full user name (required)
email	String	Unique email address (lowercase, required)
phone	String	Contact phone number (required)
password	String	Bcrypt hashed password (select: false)
role	String	'tenant' 'owner' 'admin'
status	String	'active' 'disabled'

5.2 Property Schema

Stores residential rental listings and their specifications.

Table 5.2: *Property Schema*

Field Name	Data Type	Constraints & Description
ownerId	ObjectId (ref User)	Listing property owner (required)
title	String	Property headline title
propertyType	String	'Apartment' 'House' 'Villa' 'Studio' 'Condo'
rent	Number	Monthly rental price in USD
securityDeposit	Number	Security deposit held
bedrooms / bathrooms	Number	Room and bathroom counts
amenities	[String]	List of verified amenities
status	String	'pending' 'approved' 'rejected' 'available' 'rented'
currentTenantId	ObjectId (ref User)	Active tenant when rented

5.3 RentalRequest Schema

Tracks tenant applications to owners.

Table 5.3: *RentalRequest Schema*

Field Name	Data Type	Constraints & Description
propertyId	ObjectId (ref Property)	Target rental property
tenantId	ObjectId (ref User)	Applicant tenant
ownerId	ObjectId (ref User)	Receiving landlord
message	String	Introductory cover message
status	String	'pending' 'approved' 'rejected' 'cancelled'

5.4 RentPayment Schema

Records monthly billing invoices and simulated transactions.

Table 5.4: *RentPayment Schema*

Field Name	Data Type	Constraints & Description
amount	Number	Payment invoice total in USD
dueDate	Date	Billing schedule due date
paymentDate	Date	Settlement timestamp (null if pending)
paymentMethod	String	'Credit Card' 'Debit Card' 'UPI' 'Net Banking'
transactionId	String	Unique simulated ID, e.g. TXN-DEMO-XXXXX
status	String	'pending' 'paid' 'overdue'

5.5 MaintenanceRequest Schema

Logs tenant repair tickets and their milestones.

Table 5.5: *MaintenanceRequest Schema*

Field Name	Data Type	Constraints & Description
title / description	String	Issue details and location
category	String	'Plumbing' 'Electrical' 'Internet' 'Appliance' 'Cleaning'
priority	String	'Low' 'Medium' 'High' 'Urgent'
status	String	'Pending' 'In Progress' 'Resolved'
resolutionNotes	String	Contractor actions and owner notes

5.6 Review Schema

Stores authenticated tenant ratings and feedback.

Table 5.6: *Review Schema*

Field Name	Data Type	Constraints & Description
rating	Number	1 to 5 integer rating
comment	String	Detailed written review
Index	Compound Unique	{ propertyId: 1, tenantId: 1 } (prevents duplicate reviews)

6. KEY OPERATIONAL WORKFLOWS

This chapter describes the four principal workflows that automate the rental lifecycle in Dwellio.

6.1 Listing Moderation Pipeline

When an owner submits a property, it enters the database with status 'pending' and is filtered out of all public queries. An Administrator inspects the photos and specifications at `/admin/properties`. Upon clicking 'Approve', the status updates to 'approved', instantly indexing the listing on the public catalog and landing page.

6.2 Single-Click Lease Binding

When an owner accepts an application at `/owner/requests`, the system performs the following actions automatically:

1. The application is marked 'approved'.
2. The property status is set to 'rented', and `currentTenantId` is bound to the tenant.
3. All competing pending applications for the unit are automatically rejected.
4. The first month's `RentPayment` invoice is generated for the tenant.

6.3 Simulated Rent Payment Gateway

Tenants can pay their due rent through an interactive modal with simulated card inputs and payment options. Upon confirmation, the backend generates a transaction ID (e.g. TXN-DEMO-XXXXXX), updates the status to 'paid', and automatically queues the subsequent month's invoice to keep records continuous.

6.4 Three-Step Maintenance Timeline

Maintenance complaints progress through a visually distinct status bar comprising three steps:

1. **Pending:** the request is filed by the tenant.
2. **In Progress:** a contractor is dispatched by the owner, with notes.
3. **Resolved:** completion is confirmed with resolution notes and a timestamp.

7. VERIFICATION & AUTOMATED TEST RESULTS

The system was verified through a production build, database seeding and endpoint-level tests. Table 7.1 lists each test module, the action performed, the observed output and the verdict.

Table 7.1: *Verification and Automated Test Results*

Test Module	Endpoint / Action	Observed Output	Verdict
Vite Client Build	npm run build	2,326 modules bundled with zero errors	PASSED
Database Seed	node seed.js	5 users, 8 properties, payments & reviews created	PASSED
Server Health	GET /api/health	HTTP 200 { status: 'ok' }	PASSED
Moderation Filter	GET /api/properties	Returns 7 approved listings (1 pending excluded)	PASSED
JWT Auth Login	POST /api/auth/login	Valid signed JWT token returned	PASSED
Admin Statistics	GET /api/admin/stats	KPIs: 5 Users, 8 Properties, \$10,800 Revenue	PASSED
Simulated Payment	POST /api/payments/simulate	Marked 'paid', TXN-DEMO ID generated	PASSED
Ticket Milestone	PUT /api/maintenance/:id/status	Status updated to 'In Progress' with notes	PASSED

All eight verification modules passed, confirming the correctness of the build, authentication, moderation, payment and maintenance workflows.

8. DEMO EVALUATION ACCOUNTS

The seeded database provides one account per role so that the platform can be evaluated end to end. The credentials are intended for demonstration purposes only.

Table 8.1: *Demo Evaluation Accounts*

Role	Email Address	Password	Portal Path
Administrator	admin@dwellio.com	admin123	/admin/dashboard
Property Owner	owner1@dwellio.com	owner123	/owner/dashboard
Tenant	tenant1@dwellio.com	tenant123	/tenant/dashboard

9. CONCLUSION

Dwellio (Nestora) demonstrates a production-style, resilient full-stack web application. With secure role-based navigation, automated lease binding, interactive analytics and a polished design system, it represents an exemplary reference architecture for modern PropTech rental management systems.